SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 8)

TASK 8 – EVALUATE CPU STATES AND PROTECTION

8.1 User mode and kernel mode

Modern processors provide privilege levels. A normal application runs in user mode, where it cannot execute privileged instructions or directly access protected kernel memory and device registers. The operating system kernel runs in kernel mode, where it can configure hardware, manage memory mappings, handle interrupts and perform privileged operations. A hardware mode/privilege indicator and memory-protection mechanisms enforce the boundary.

Privileged instructions include operations such as changing interrupt-control state, configuring memory-management hardware and performing certain I/O operations. An application must request these operations through an authorized system-call interface rather than executing them directly.

8.2 Interrupts, traps and timer interrupts

Event	Meaning	Smart Hospital relevance
Hardware interrupt	An asynchronous signal from a device or timer that asks the CPU to handle an event.	A sensor interface, network controller or timer can notify the processor that work is ready.
Trap / exception	A synchronous event caused by the current instruction or an intentional software request.	A system-call instruction enters the kernel; an invalid memory access can raise an exception.
Timer interrupt	A periodic or one-shot timer event used by the OS to regain control and manage scheduling.	Allows preemption so a low-priority task cannot occupy the CPU indefinitely and higher-priority alarm work can run.

The timer interrupt is important even when the system is busy. It allows the kernel to regain CPU control and apply scheduling decisions. In a real-time monitor, the highest-priority alarm task must be able to pre-empt lower-priority work, and interrupt handlers must be short and bounded.

8.3 What if any program could disable interrupts?

If an ordinary application could disable interrupts and leave them disabled, the OS might not receive timer interrupts or device notifications. The scheduler could fail to regain control, sensor events could be delayed, and a low-priority or faulty task could block the alarm path. Such a design could prevent the monitor from meeting its 10 ms requirement and could make the whole device unresponsive. Therefore, interrupt control must remain a privileged operation, and the kernel must keep interrupt-disabled critical sections short.

8.4 How hardware protects process memory

The memory management unit (MMU) uses page tables and access permissions to map each process's virtual address space. A process can access only pages marked as accessible to it. Kernel pages are protected from user-mode access, and pages belonging to another process are not mapped as readable/writable by the current process. An invalid access raises a protection exception, which the OS handles, often by terminating the offending process or delivering a signal.

8.5 Alternative: polling instead of interrupts

With polling interval T, an event arriving just after a check waits almost T; the average wait is about T/2. An 8 ms poll can consume most of the 10 ms alarm budget before conversion, scheduling and output. Faster polling wastes CPU cycles and power. Interrupt-driven I/O avoids repeated idle checks, but interrupt latency and handler/sch